Benchmarks from Van der Waerden Problems and Integer Multiplication Verification

Shimin Zhang
School of Computer Science, Georgia Institute of Technology, Atlanta, Georgia, USA
simetra.msz@gmail.com

Abstract

We present two families of SAT benchmarks for the SAT Competition 2026. The first family encodes the *Van der Waerden (VDW) problem*: can we k-color $\{1, \ldots, n\}$ such that no color class contains an arithmetic progression of the prescribed length? The second family encodes *integer factorization*: given target N, does a non-trivial n-bit factorization $a \cdot b = N$ with $a, b > 1$ exist? Composite targets yield satisfiable instances; prime targets yield unsatisfiable ones. Both families span a range of difficulty levels and are fully generated from open-source Python scripts included in the submission.

1. Van der Waerden Benchmarks

1.1. Background

Van der Waerden's theorem [1] states that for any $k \geq 1$ and positive integers $\ell_1, \ldots, \ell_k$, there exists a finite threshold $W(k; \ell_1, \ldots, \ell_k)$ such that every k-coloring of $\{1, \ldots, n\}$ with $n \geq W$ contains a monochromatic arithmetic progression (AP) of length ℓ_i in some color i. Determining exact Van der Waerden numbers has been a prominent benchmark problem in SAT solving [**?**, **?**], producing challenging instances at the SAT/UNSAT threshold.

1.2. Encoding

We encode the decision problem "Can we k-color $\{1, \ldots, n\}$ without monochromatic APs of prescribed lengths?" as a CNF formula. Variables $x_{i,c}$ indicate that position $i \in \{1, \ldots, n\}$ receives color $c \in \{0, \ldots, k-1\}$.

Exactly-one-color clauses. For each position i, at-least-one ($\bigvee_c x_{i,c}$) and at-most-one ($\neg x_{i,c_1} \vee \neg x_{i,c_2}$ for all $c_1 < c_2$) clauses are added.

No-AP clauses. For each color c, step $d \geq 1$, and start position a such that $a + (\ell_c - 1)d \leq n$, we add the clause forbidding the AP from being entirely color c: $\neg x_{a,c} \vee \neg x_{a+d,c} \vee \cdots \vee \neg x_{a+(\ell_c-1)d,\,c}$.

1.3. Instances

We provide 10 instances at known Van der Waerden thresholds (Table 1), combining symmetric 2-color cases $W(2; k, k)$ and asymmetric cases $W(2; \ell_1, \ell_2)$. The satisfiable instances have $n = W - 1$; the unsatisfiable instances have $n = W$ (exactly at the threshold). The $W(2; 5, 5) = 178$ pair offers the most demanding benchmarks in this family.

Table 1: Van der Waerden Instances

Params	n	Vars	Clauses	SAT?
$W(2;3,3) = 9$	8	16	40	✓
	9	18	50	×
$W(2;4,4) = 35$	34	68	420	✓
	35	70	444	×
$W(2;5,5) = 178$	177	354	8010	✓
	178	356	8100	×
$W(2;3,7) = 46$	45	90	721	✓
	46	92	752	×
$W(2;3,8) = 58$	57	114	1102	✓
	58	116	1140	×

2. Integer Multiplication Benchmarks

2.1. Background

Verifying hardware multipliers is a central application of SAT and computer algebra [**?**]. The integer factorization problem—given N, find $a, b > 1$ with $a \cdot b = N$— provides a natural source of hard SAT instances: satisfiable when N is composite (a witness factorization exists), and unsatisfiable when N is prime (no non-trivial factorization exists). The hardness scales with the bit-width n of the factors, making this family suitable for testing solver performance across a range of difficulty levels.

2.2. Encoding

We encode n-bit unsigned multiplication via a Tseitin transformation [**?**] of a schoolbook partial-product multiplier. Input bits $a_0, \ldots, a_{n-1}$ and $b_0, \ldots, b_{n-1}$ (LSB first) are the free variables.

Partial products. Gate $pp_{i,j} = a_i \wedge b_j$ is encoded with the standard 3-clause AND Tseitin encoding.

Adder tree. Partial products are reduced column-by-column. Column k accumulates all bits of weight 2^k using half-adders and full-adders (encoded via standard XOR/AND Tseitin gates), producing a single sum bit p_k and carries forwarded to column $k+1$.

Output and non-triviality constraints. Product bits $p_0, \ldots, p_{2n-1}$ are fixed to N via unit clauses. Non-triviality is enforced by adding $\bigvee_i a_i$ (not zero) and $\neg a_0 \vee \bigvee_{i>0} a_i$ (not one), and symmetrically for b.

2.3. Instances

We provide 10 instances across four bit-widths (Table **??**). UNSAT instances use prime targets; SAT instances use perfect squares of primes to ensure exactly-n-bit factors exist. The $n = 16$ and $n = 20$ instances are expected to be the hardest.

Table 2: Integer Multiplication Instances

n	Target N	Vars	Clauses	SAT?
8	65003 (prime)	336	1084	×
8	$241^2 = 58081$	336	1084	✓
8	$239^2 = 57121$	336	1084	✓
8	63499 (prime)	336	1084	×
12	4194319 (prime)	792	2584	×
12	$4093^2 = 16752649$	792	2584	✓
16	$\approx 2^{32}$ prime	1440	4724	×
16	65535^2	1440	4724	✓
20	$\approx 2^{39}$ prime	2280	7504	×
20	$1023^2 = 1046529$	2280	7504	✓

3. Availability and Generation

Both benchmark families are generated by open-source Python scripts (`gen_vdw.py` and `gen_mult.py`) provided in the submission archive alongside the 20 CNF instances in DIMACS format. The generators accept configurable parameters, enabling users to produce additional instances of arbitrary size. The VDW generator takes k, the AP lengths $\ell_1, \ldots, \ell_k$, and the domain size n. The multiplication generator takes the bit-width n and target value N.

Acknowledgments

This work was supported by the LLM-based SAT research project at Georgia Tech.

References

[1] B. L. van der Waerden, "Beweis einer Baudetschen Vermutung," *Nieuw Arch. Wiskunde*, vol. 15, pp. 212–216, 1927.

[2] M. Kouril and J. L. Paul, "The Van der Waerden Number $W(2;3,3)$ Is 9," *Experimental Mathematics*, vol. 17, no. 1, pp. 53–61, 2008.

[3] T. Ahmed, "On the Van der Waerden Numbers $w(2;3,t)$," *Integers*, vol. 12, no. 5, pp. 855–875, 2012.

[4] A. Biere, M. Heule, H. van Maaren, and T. Walsh, *Handbook of Satisfiability*, 2nd ed., IOS Press, 2021.

[5] G. S. Tseitin, "On the complexity of derivation in propositional calculus," in *Studies in Constructive Mathematics and Mathematical Logic*, A. O. Slisenko, Ed., pp. 115–125, 1968.